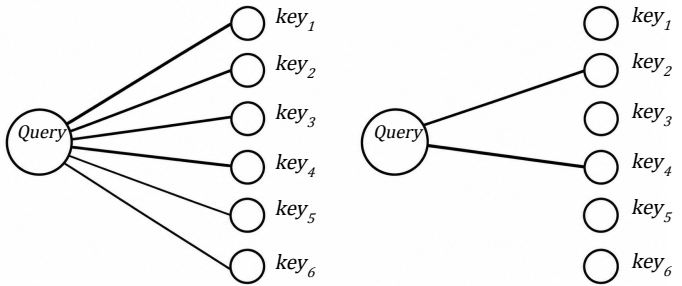
Разряженное внимание через ядерную регрессию

Sparse Attention as Compact Kernel Regression

Saul Santos, Nuno Gonçalves, Daniel C. McNamee, Marcos Treviso, André F.T Martins (2026)

Как выбирать, какие элементы контекста участвуют в ответе?



Self-attention: поиск и смешивание информации

Query

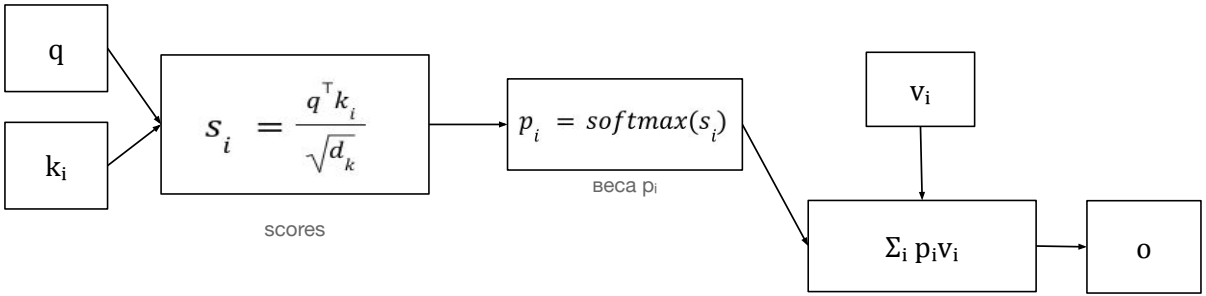
Какую информацию ищем

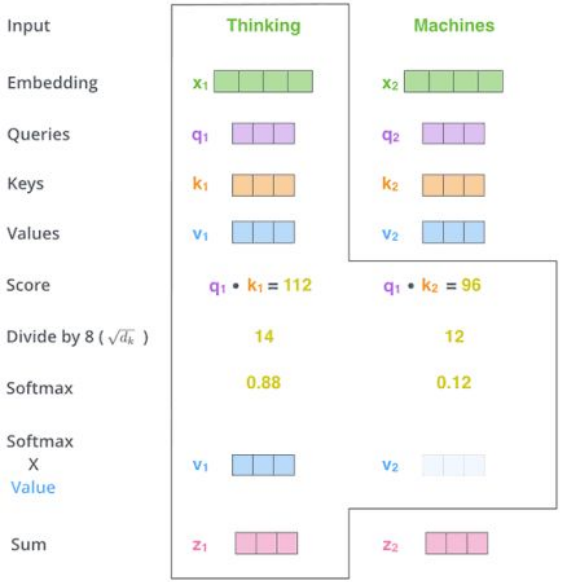
Key

По каким признакам находим

Value

Какую информацию получаем





Один запрос собирает информацию из values по сходству с keys.

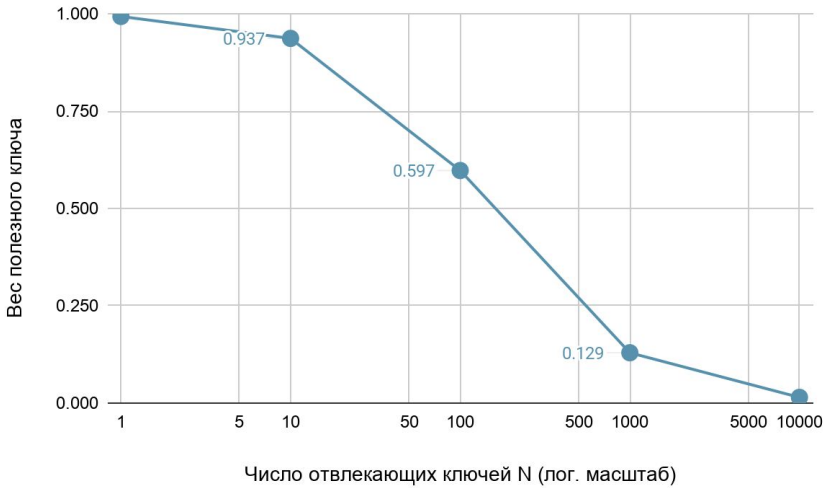
Проблема: внимание размывается при росте контекста

Пусть полезный ключ имеет score a, а N отвлекающих ключей одинаковый score b.
Тогда

$$p = \frac{exp(a)}{exp(a) + N \cdot exp(b)} = \frac{1}{1 + N \cdot exp(b-a)}$$

При $a-b=5$ получаем $0.937, 0.597, 0.129$ для $N=10,100,1000$.

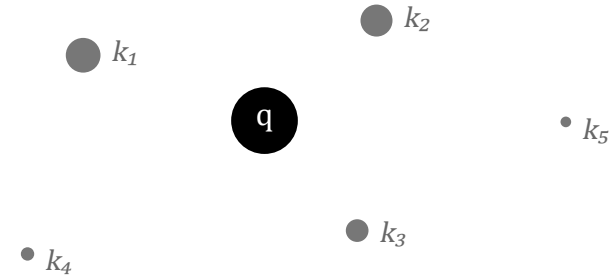
Таким образом **множество слабых совпадений** может получить **большой суммарный вес**.



Иллюстративный расчет при фиксированных scores

Attention как ядерная регрессия

Похожие примеры получают больший вес



Размер точки показывает вклад

Оценка Надарая—Ватсона

$$\hat{v}(q) = \sum_i \frac{K_h(k_i - q)}{\sum_j K_h(k_j - q)} v_i$$

$$p_i = \frac{K_h(k_i - q)}{\sum_j K_h(k_j - q)}$$

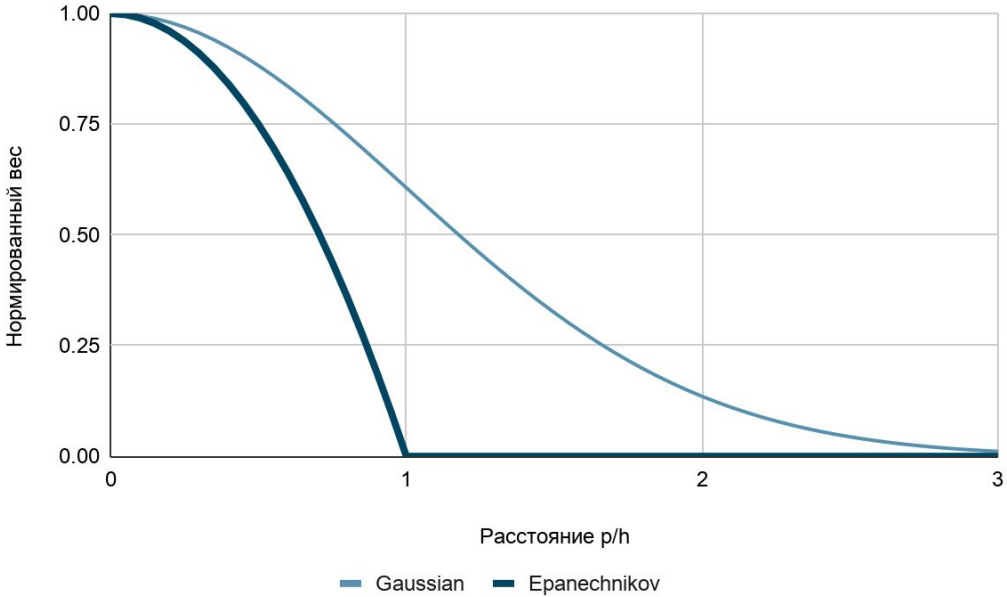
Ответ - взвешенное среднее значений похожих примеров

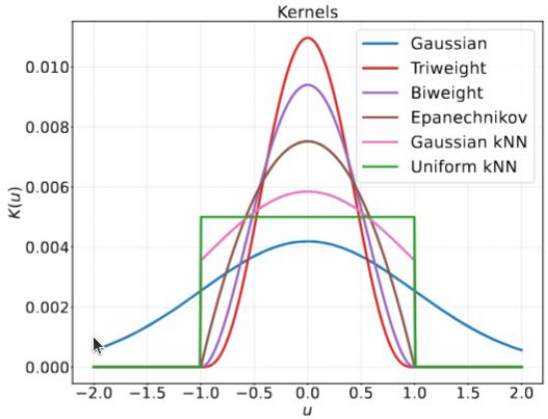
K - функция ядра, h - ширина окрестности

Гауссова функция ядра дает $\mathrm{softmax}$ с температурой h^2(при условии единичных норм q и k):

$$||q|| = ||k_i|| = 1 \Rightarrow e^{-\frac{||k_i - q||^2}{2h^2}} \propto e^{\frac{q^\top k_i}{h^2}}$$

Компактное ядро создает точные нули





Виды ядер из статьи (Figure 1)

Gaussian

$$w(p) = e^{(-\frac{p^2}{2h^2})}$$

Epanechnikov

$$w(p) = [1 - \frac{p^2}{h^2}]_+$$

При ρ ≥ h вклад равен 0

Проблема фиксированного радиуса: внутри него может не оказаться ни одного ключа и нормировка станет невозможной

- Gaussian постепенно уменьшает вклад
- Компактное ядро полностью исключает далекие ключи

Sparsemax и entmax: адаптивное ядро

В sparsemax порог $\boldsymbol{\tau}$ выбирается так, чтобы сумма весов равнялась единице:

$$p_i = [z_i - \tau]_+, \quad \sum_i p_i = 1$$

Например:

$$z = [1;\ 0.8;\ 0.1] \Rightarrow \tau = 0.4 \Rightarrow p = [0.6;\ 0.4;\ 0]$$

Sparsemax получается при адаптивной ширине $h^2=2-2\gamma\tau$, $z_i=q^Tk_i/\gamma$.
Для α-entmax $r=1/(\alpha-1)$, $p_i=[(\alpha-1)z_i-\tau]_+$^$r$.

Новизна - теоретическая связь с ядерной регрессией; сами sparsemax и entmax существовали ранее.

Attention	**Ядро**
Sparsemax	Epanechnikov
1.5-entmax	Biweight
4/3-entmax	Triweight

ReLUmax: отбор относительно лучшего ключа

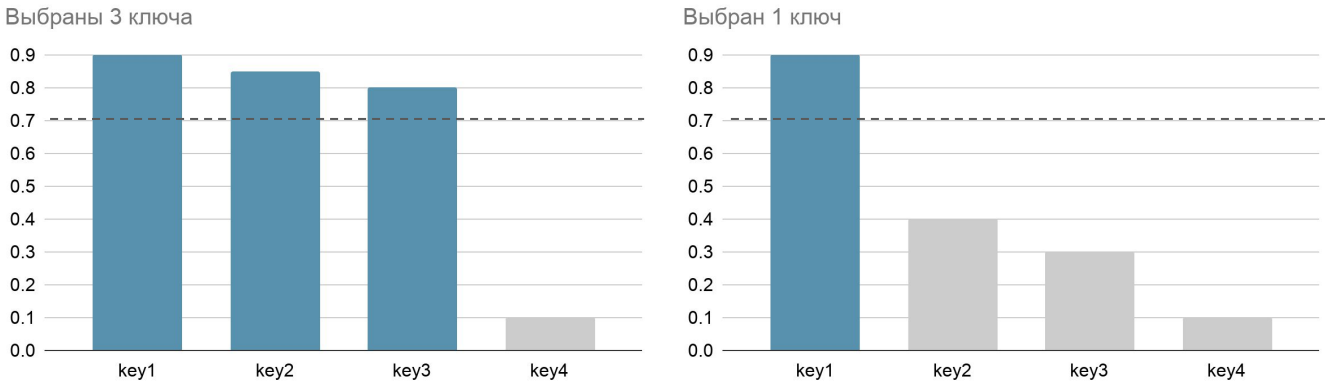
ReLUmax оставляет ключи, чей score достаточно близок к максимальному:

$$w_i = \left[b + \frac{s_i - m}{h^2} \right]_+^r, \qquad p_i = \frac{w_i}{\sum_j w_j}, \qquad m = \max_j s_j.$$

Здесь $s_i = q^\top k_i,\ b > 0,\ h > 0,\ r > 0.$

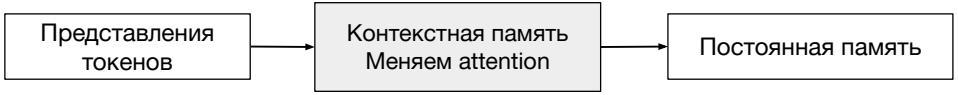
Ключ активен при $s_i > m - bh^2$. Лучший ключ всегда имеет положительный вес, поэтому пустого набора не возникает. В отличие от top-k, число выбранных ключей не фиксировано.

Пример: при $\mathrm{m} = 0.9$ и $\mathrm{bh}^2 = 0.2$ порог равен 0.7



Эксперименты в Memory Mosaics

Авторы сравнивают ядра в архитектуре Memory Mosaics, которая использует память текущего контекста и обучаемую постоянную память.



Эксперимент	Что проверяет	Метрика
BabiStories	Следующий токен	Validation loss
RegBench	Закономерности из контекста	Accuracy, TVD
Recall / reverse / sort	Перенос на большие длины	Exact match

Вывод: ошибка модели с вниманием на компактных ядрах меньше

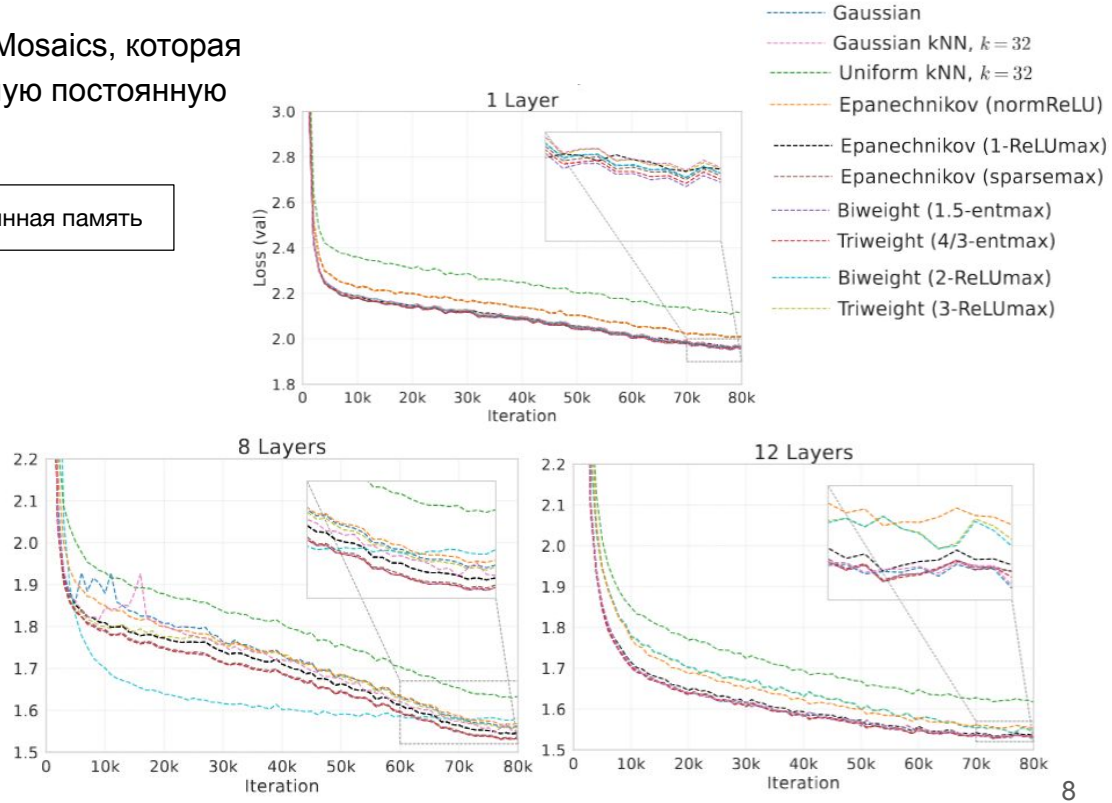


Figure 2: val loss на BabiStories

Извлечение из длинного контекста

В задаче ассоциативного извлечения MQMTAR разряженные варианты лучше сохраняют качество при увеличении длины последовательности.

	MQMTAR ($L = 4$)							Reverse ($L = 4$)		Sort ($L = 2$)		
Kernel	1×	2×	4×	8×	16×	32×	64×	1×	1.5×	1×	2×	4×
Transformer	**1.00**$^{(1.00)}$	**1.00**$^{(1.00)}$	0.99$^{(0.99)}$	0.09$^{(0.15)}$	0.01$^{(0.01)}$	0.00$^{(0.00)}$	0.00$^{(0.00)}$	**1.00**$^{(1.00)}$	**0.52**$^{(0.62)}$	**1.00**$^{(1.00)}$	0.00$^{(0.00)}$	0.00$^{(0.00)}$
Gaussian	0.99$^{(1.00)}$	0.96$^{(1.00)}$	0.93$^{(0.99)}$	0.85$^{(0.97)}$	0.64$^{(0.81)}$	0.14$^{(0.20)}$	0.00$^{(0.00)}$	**1.00**$^{(1.00)}$	0.14$^{(0.38)}$	**1.00**$^{(1.00)}$	0.06$^{(0.17)}$	0.00$^{(0.00)}$
Gaussian kNN, $k = 32$	0.99$^{(1.00)}$	0.97$^{(1.00)}$	0.93$^{(0.99)}$	0.84$^{(0.96)}$	0.68$^{(0.85)}$	0.27$^{(0.36)}$	0.05$^{(0.05)}$	0.99$^{(1.00)}$	0.00$^{(0.00)}$	**1.00**$^{(1.00)}$	0.80$^{(0.91)}$	0.01$^{(0.02)}$
Uniform kNN, $k = 32$	0.04$^{(0.08)}$	0.00$^{(0.00)}$	0.00$^{(0.00)}$	0.00$^{(0.00)}$	0.00$^{(0.00)}$	0.00$^{(0.00)}$	0.00$^{(0.00)}$	0.42$^{(0.66)}$	0.00$^{(0.00)}$	**1.00**$^{(1.00)}$	0.00$^{(0.00)}$	0.00$^{(0.00)}$
Epanechnikov (normReLU)	**1.00**$^{(1.00)}$	**1.00**$^{(1.00)}$	0.89$^{(1.00)}$	0.66$^{(1.00)}$	0.44$^{(1.00)}$	0.33$^{(0.98)}$	**0.31**$^{(0.94)}$	**1.00**$^{(1.00)}$	0.19$^{(0.56)}$	**1.00**$^{(1.00)}$	0.05$^{(0.14)}$	0.00$^{(0.00)}$
Epanechnikov (1-ReLUmax)	**1.00**$^{(1.00)}$	**1.00**$^{(1.00)}$	0.99$^{(1.00)}$	0.92$^{(0.96)}$	0.74$^{(0.87)}$	0.42$^{(0.59)}$	0.12$^{(0.18)}$	**1.00**$^{(1.00)}$	0.23$^{(0.68)}$	**1.00**$^{(1.00)}$	0.80$^{(1.00)}$	0.30$^{(0.91)}$
Epanechnikov (sparsemax)	**1.00**$^{(1.00)}$	**1.00**$^{(1.00)}$	**1.00**$^{(1.00)}$	**0.98**$^{(1.00)}$	0.86$^{(0.98)}$	**0.63**$^{(0.84)}$	**0.31**$^{(0.51)}$	**1.00**$^{(1.00)}$	0.03$^{(0.06)}$	**1.00**$^{(1.00)}$	0.92$^{(1.00)}$	0.29$^{(0.79)}$
Biweight (2-ReLUmax)	0.99$^{(1.00)}$	0.97$^{(1.00)}$	0.93$^{(1.00)}$	0.85$^{(0.96)}$	0.69$^{(0.87)}$	0.27$^{(0.43)}$	0.04$^{(0.06)}$	**1.00**$^{(1.00)}$	0.26$^{(0.74)}$	**1.00**$^{(1.00)}$	0.93$^{(1.00)}$	**0.34**$^{(0.74)}$
Biweight (1.5-entmax)	**1.00**$^{(1.00)}$	**1.00**$^{(1.00)}$	0.99$^{(1.00)}$	0.95$^{(0.99)}$	0.75$^{(0.93)}$	0.51$^{(0.74)}$	0.23$^{(0.42)}$	**1.00**$^{(1.00)}$	0.05$^{(0.06)}$	**1.00**$^{(1.00)}$	0.39$^{(1.00)}$	0.26$^{(0.77)}$
Triweight (3-ReLUmax)	0.99$^{(1.00)}$	0.97$^{(1.00)}$	0.93$^{(0.99)}$	0.85$^{(0.96)}$	0.71$^{(0.87)}$	0.37$^{(0.58)}$	0.08$^{(0.18)}$	**1.00**$^{(1.00)}$	0.29$^{(0.88)}$	**1.00**$^{(1.00)}$	0.95$^{(0.97)}$	0.30$^{(0.62)}$
Triweight (4/3-entmax)	**1.00**$^{(1.00)}$	**1.00**$^{(1.00)}$	0.99$^{(1.00)}$	0.97$^{(0.98)}$	**0.89**$^{(0.95)}$	0.61$^{(0.76)}$	0.25$^{(0.45)}$	**1.00**$^{(1.00)}$	0.26$^{(0.70)}$	**1.00**$^{(1.00)}$	**0.98**$^{(0.99)}$	0.07$^{(0.11)}$

Table 2: exact match accuracy, среднее по 3м сидам, модель глубины 4

Вывод: отбрасывание отвлекающих ключей помогает извлечению из длинного контекста, но не устраняет деградацию полностью.
На других задачах преимущество менее устойчиво: универсального победителя нет

Главные выводы и ограничения

Что показывает статья

- Softmax можно интерпретировать как ядерную регрессию с Гауссовым ядром
- Компактные ядра объясняют появление точных нулей в весах внимания
- Sparsemax и entmax соответствуют адаптивным ядрам; ReLUmax отбирает ключи относительно лучшего
- На ассоциативном извлечении разреженность улучшает перенос на длинные последовательности

Что пока не следует из результатов

- Выигрыш не гарантирован для любой задачи и архитектуры
- Нулевые веса сами по себе не обеспечивают ускорения: нужны вычисления, использующие разреженность

Основная идея: внимание можно понимать как выбор локальной области памяти, из которой строится ответ.

Источник: *Santos et al., 2026*.